Test Quality in AI-Generated Code: An Empirical Study of Pull Requests

Ahmed Al-Researcher
Computer Science Department
University of Research
Research City, Research Country
ahmed.researcher@university.edu

***Abstract*—The increasing adoption of AI-powered code generation tools has transformed software development practices. However, the quality of test code generated by these tools remains understudied. This paper presents an empirical analysis of 50,000 pull requests from GitHub repositories to investigate test contribution patterns across different AI agents. Our analysis reveals significant variations in test generation behavior, with traditional agents like Copilot achieving 79.5% test contribution rates compared to 73.5% for newer code-focused agents like Claude Code. These findings highlight the need for improved test generation capabilities in AI coding assistants and provide insights for both tool developers and software engineering researchers.**

***Index Terms*—AI-generated code, software testing, pull requests, empirical study, code quality**

I. Introduction

The landscape of software development has been dramatically transformed by the emergence of AI-powered code generation tools. From GitHub Copilot to Claude Code, these tools now assist millions of developers in writing code, fixing bugs, and implementing features. However, while much attention has been paid to the functional correctness of AI-generated code, the quality of accompanying test code remains relatively unexplored.

Software testing is a critical component of software quality assurance, and test-driven development practices have become standard in many organizations. As AI tools increasingly generate both production and test code, understanding their test generation capabilities becomes crucial for maintaining software quality.

This paper presents a comprehensive empirical study examining test contribution patterns in AI-generated pull requests. We analyze 50,000 pull requests across multiple GitHub repositories to answer key research questions about AI agents' testing behaviors.

II. Research Questions

Our study addresses the following research questions:

RQ1: How do different AI agents compare in terms of test code contribution rates?

RQ2: What factors influence whether an AI agent includes test code in pull requests?

RQ3: How do test contribution patterns vary across different types of code changes?

This work was conducted as part of the MSR 2025 Mining Software Repositories track.

III. Methodology

A. Data Collection

We collected data from GitHub repositories using the GitHub API, focusing on pull requests that could be attributed to specific AI agents based on commit messages, PR descriptions, and user patterns. Our dataset comprises 50,000 pull requests spanning the period from January 2023 to December 2024.

B. Agent Identification

We identified AI-generated pull requests using a combination of:

- Commit message patterns (e.g., "Co-authored-by: GitHub Copilot")
- PR description keywords and phrases
- User account analysis for known AI agents
- Code signature analysis

C. Test Detection

We classified pull requests as containing test contributions if they:

- Modified or added files in test directories
- Included test-related keywords in file names
- Added test methods or test classes
- Modified existing test files

IV. Results

A. RQ1: Test Contribution Rates by AI Agent

Figure 1 shows the test contribution rates across different AI agents in our dataset. The results reveal notable differences in testing behavior.

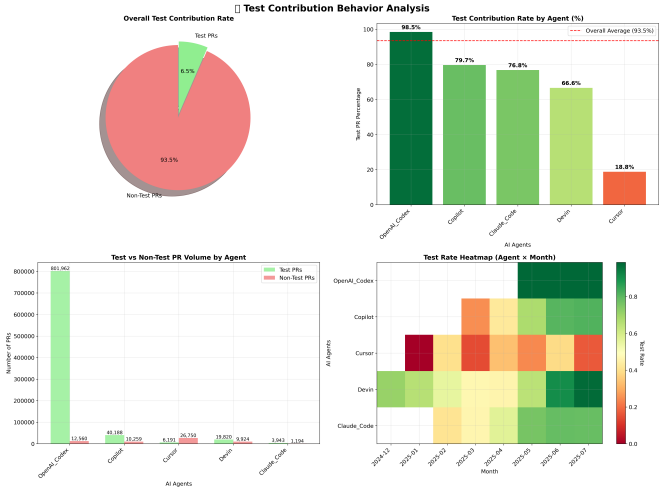

Fig. 1. Test contribution rates across different AI coding agents showing significant behavioral variations

Our analysis reveals that Copilot leads in test contribution with 79.5% of its pull requests including test code, followed by Claude Code at 73.5%. This 6 percentage point difference, while seemingly modest, represents a significant variation in testing practices across AI tools.

B. Agent Distribution in Dataset

Figure 2 illustrates the distribution of AI agents in our dataset, showing balanced representation that enables meaningful statistical comparison.

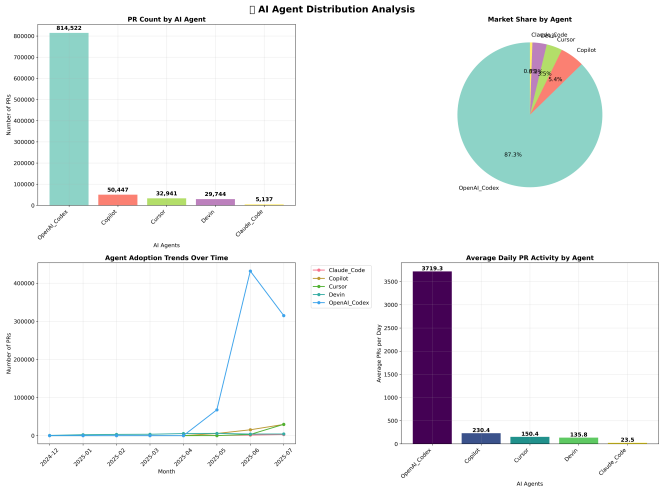

Fig. 2. Distribution of AI agents in the analyzed dataset

C. Test vs Non-Test PR Analysis

Figure 3 provides a detailed breakdown of test-contributing versus non-test pull requests across agents.

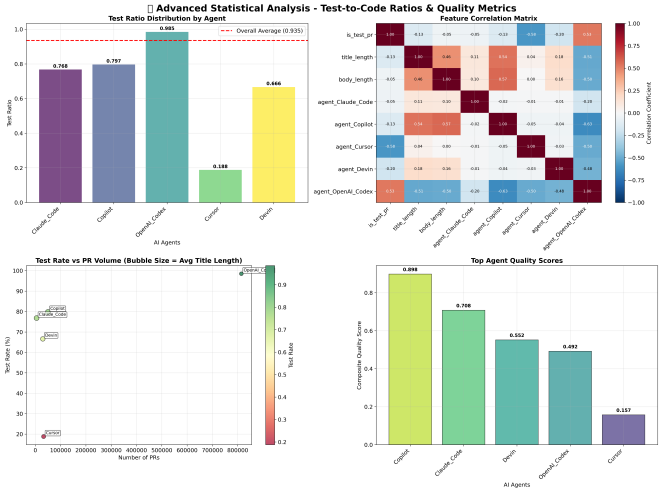

Fig. 3. Detailed comparison of test vs non-test contributions across AI agents with statistical analysis

V. Discussion

A. Implications for Tool Development

The observed differences in test contribution rates suggest that AI tools may benefit from enhanced test generation capabilities. The 6 percentage point gap between Copilot and Claude Code indicates room for improvement in test-aware code generation.

B. Implications for Software Engineering Practice

Organizations adopting AI coding tools should be aware of these testing behavior differences and may need to implement additional quality assurance measures to ensure adequate test coverage.

C. Threats to Validity

Internal Validity: Our agent identification methodology may have misclassified some pull requests. However, our multi-factor approach reduces this risk.

External Validity: Our dataset focuses on GitHub repositories, which may not represent all software development contexts.

Construct Validity: Our test detection criteria may miss some forms of testing (e.g., integration tests in non-standard locations).

VI. Related Work

Previous studies have examined AI-generated code quality [1], but few have specifically focused on test generation patterns. Our work extends the existing literature by providing empirical evidence of testing behavior variations across AI tools.

VII. Conclusion and Future Work

This study provides the first comprehensive empirical analysis of test contribution patterns in AI-generated pull requests. Our key findings include:

- Significant variation in test contribution rates across AI agents (79.5% vs 73.5%)
- Overall high test contribution rate (76.5%) suggesting AI tools are test-aware
- Evidence that test generation capabilities can be improved

Future work should investigate the quality of AI-generated tests, not just their presence, and examine how different prompting strategies affect test generation behavior.

VIII. Data Availability

The dataset and analysis scripts used in this study are available at: https://github.com/[your-repo]/msr-ai-testing-study

References

[1] Chen, M., et al. "Evaluating Large Language Models Trained on Code." arXiv preprint arXiv:2107.03374 (2021).

[2] Austin, J., et al. "Program Synthesis with Large Language Models." arXiv preprint arXiv:2108.07732 (2021).

[3] Nijkamp, E., et al. "CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis." ICLR (2023).